

cannot be used by the foreign program itself, which may require rewriting it to avoid this register.

The code works as follows: First the target address under inspection is copied to a scratch register, S1. Then this register is shifted right precisely the correct number of bits to isolate the common prefix in S1. Next the isolated prefix is compared to the correct prefix initially loaded into S2. If they do not match, a trap occurs and the foreign program is killed. This code sequence requires four instructions and two scratch registers.

Patching the binary program during execution requires some work, but it is doable. It would be simpler if the foreign program were presented in source form and then compiled locally using a trusted compiler that automatically checked the static addresses and inserted code to verify the dynamic ones during execution. Either way, there is some runtime overhead associated with the dynamic checks. Wahbe et al. (1993) have measured this as about 4%, which is perhaps not too bad.

A second problem that must be solved is what happens when a foreign program tries to make a system call. The solution here is straightforward. The system-call instruction is replaced by a call to a special module called a **reference monitor** on the same pass that the dynamic address checks are inserted (or, if the source code is available, by linking with a special library that calls the reference monitor instead of making system calls). Either way, the reference monitor examines each attempted call and decides if it is safe to perform. If the call is deemed acceptable, such as writing a temporary file in a designated scratch directory, the call is allowed to proceed. If the call is known to be dangerous or the reference monitor cannot tell, the foreign program is killed. If the reference monitor can tell which foreign program called it, a single reference monitor somewhere in memory can handle the requests from all foreign programs. The reference monitor normally learns about the permissions from a configuration file.

9.9 RESEARCH ON SECURITY

Few topics see more activity than security in operating systems. Research is taking place in all areas: cryptography, attacks, defenses, compilers, hardware, formal methods, etc. A more-or-less continuous stream of high-profile security incidents ensures that research interest in security, both in academia and in industry, is not likely to waver in the next few years either.

Even venerable concepts such as passwords are still active research topics, for instance when modeling the strength of a password (Pasquini et al., 2021) or protecting password vaults as used by password managers using a set of decoy vaults (Cheng et al., 2021). Also other authentication factors, such as security keys, are targeted by security researchers to probe for weaknesses (Roche et al., 2021).

Bugs in the operating system itself are clearly a major concern and researchers are developing testing frameworks for various operating systems to find and

classify bugs better, e.g., on Linux (Lin et al., 2022) and Windows (Choi et al., 2021). Others implement techniques to detect memory errors and manifestations of undefined behavior by instrumenting the code with all manner of checks—an approach that became especially popular after the release of Google’s AddressSanitizer (Serebryany, 2013). As an alternative, some projects try to formally verify that the OS is free from certain classes of bugs (Klein et al., 2009; Yu et al., 2021).

In spite of what we see in the movies, kernel exploitation is not easy. Often the vulnerabilities build on complex heap overflows which require that objects are aligned in memory exactly right. Fortunately for the attacker, major advances are made in the automation of the exploitation process. For instance, researchers have developed techniques to manipulate the memory layout in the kernel to automatically obtain the desired alignment of objects (Chen and Xing, 2019).

Protecting sensitive state is challenging especially with low overhead, but sometimes hardware features help. An example is the recent wave of use-cases for **MPK (Memory Protection Keys)**, a hardware feature already present in the IBM 360 more than half a century ago, but only recently added to Intel’s x86-64 processor architecture. Researchers have shown that MPKs can be used to isolate sensitive state very efficiently (Vahldiek-Oberwagner, 2019). Hardware assistance is not the only way. Another way of protecting sensitive state is by hiding it at a random location in the enormous 64-bit address space, and maybe even throw in continuous re-randomization for good measure (Wang et al., 2019).

Capabilities and access control are still active research areas. Here also, sometimes hardware features breathe new life in these old research topics. For instance (Davis et al., 2019) describe how hardware-supported capabilities can help to provide very powerful access control and restrictions.

A topic that has been all the rage since the original Meltdown and Spectre vulnerabilities were disclosed is transient execution attacks (Xiong and Szefer, 2021). Unfortunately, there is a myriad of root causes for transient execution (Ragab et al., 2021) and many of them may lead to exploitable vulnerabilities. Not surprisingly, there are many efforts to remedy the situation, for instance using software mitigations or formal methods (Duta et al., 2021; and Loughlin et al., 2019).

Other hardware-related issues concern malicious devices performing DMA and accessing memory beyond the operating system’s control (Markettos, 2019), even in the presence of an MMU for such devices. Again, automatically finding and analyzing DMA-related issues is important to fix the vulnerabilities before the attackers can exploit them (Alex et al., 2019).

While we are on the topic of hardware: recognizing the threats posed by attackers, many devices nowadays have security features. The question is how good they are. For instance, many SSDs have encryption built in. Unfortunately, just because you have encryption does not mean that you are secure. For example, Meijer and Van Gastel, 2019 show that the additional security guarantees provided by such devices are often zero. Unfortunately, also this is a fairly common thing to happen. Getting security right in the face of determined attackers is really hard.

9.10 SUMMARY

Computers frequently contain valuable and confidential data, including tax returns, credit card numbers, business plans, trade secrets, and much more. The owners of these computers are usually quite keen on having them remain private and not tampered with, which rapidly leads to the requirement that operating systems must provide good security. Security comprises confidentiality, integrity, and availability. To develop secure systems, we consistently apply security principles. The structure of an operating system matters for its security properties, as some designs (e.g., monolithic systems) make it difficult to apply principles such as the Principle Of Least Authority. In general, the security of a system is inversely proportional to the size of the trusted computing base.

A fundamental component of security for operating systems concerns access control to resources. Access rights to information can be modeled as a big matrix, with the rows being the domains (users) and the columns being the objects (e.g., files). Each cell specifies the access rights of the domain to the object. Since the matrix is sparse, it can be stored by row, which becomes a capability list saying what that domain can do, or by column, in which case it becomes an access control list telling who can access the object and how. Using formal modeling techniques, information flow in a system can be modeled and limited. However, sometimes it can still leak out using covert channels, such as modulating CPU usage.

Security properties that rest on strong proofs and formalizations go beyond the modeling of information flow and include cryptography. Cryptographic schemes can be categorized as secret key or public key. A secret-key method requires the communicating parties to exchange a secret key in advance, using some out-of-band mechanism. Public-key cryptography does not require secretly exchanging a key in advance, but it is much slower in use. Sometimes it is necessary to prove the authenticity of digital information, in which case cryptographic hashes, digital signatures, and certificates signed by a trusted certification authority can be used.

In any secure system, users must be authenticated. This can be done by something the user knows, something the user has, or something the user is (biometrics). Two-factor identification, such as an iris scan and a password, can be used to enhance security.

Many kinds of bugs in software can be exploited to take over programs and systems. These include buffer overflow vulnerabilities, format-string bugs, use-after-free, type confusion bugs, null pointer dereferences, double frees, integer overflows, and several others. They enable a variety of attacks, nowadays frequently reusing the original program code to implement the malicious behavior. To deal with such vulnerabilities, vendors deploy defenses such as stack canaries, data execution prevention, and address-space layout randomization.

Insiders, such as company employees, can defeat system security in different ways. These include logic bombs set to go off on some future date, trap doors to allow the insider unauthorized access later, and login spoofing.